st440hw5rmark

Logan Moy

2025-03-21

Jags

3.9

```
 # Y is num of home runs
 data <- list(regY = 563, regN = 2820, worldY = 10, worldN = 27)

 model_string <- textConnection("model{

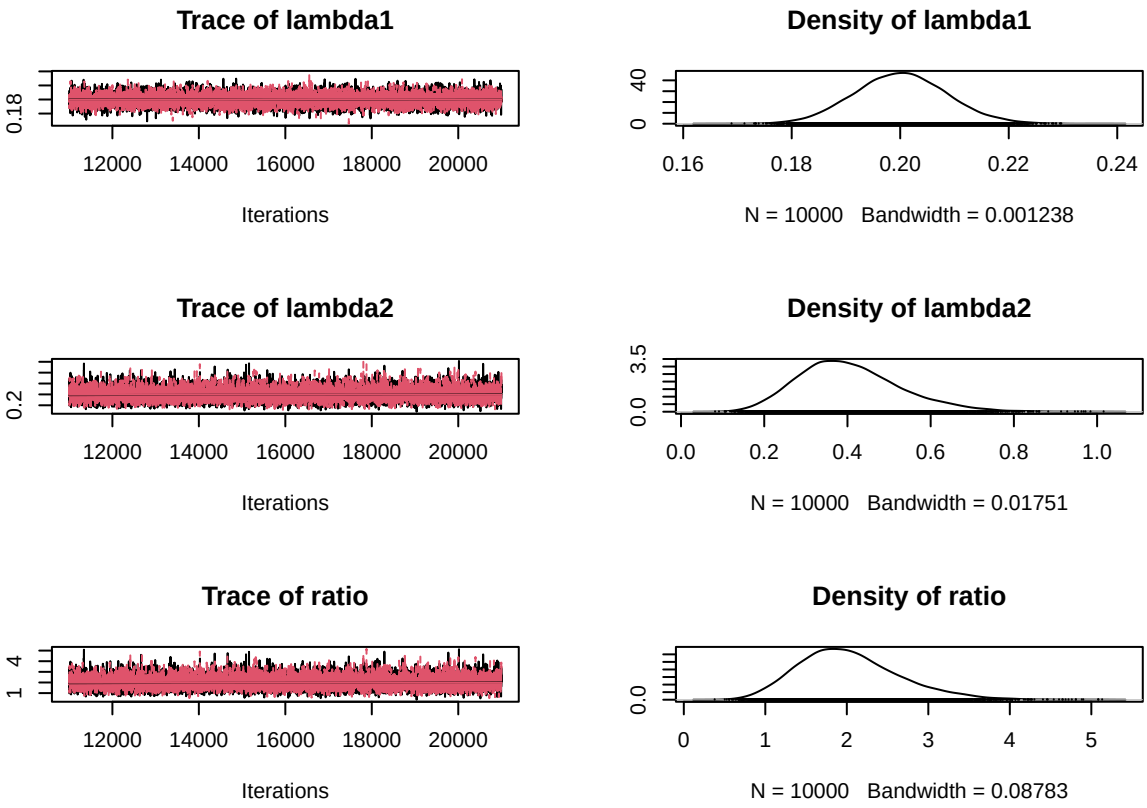
  # Likelihood
   regY ~ dpois(regN * lambda1)
   worldY ~ dpois(worldN * lambda2)

  # Priors
   lambda1 ~  dunif(0, 10)
   lambda2 ~  dunif(0, 10)
   ratio <- lambda2/lambda1
}")

inits <- list(lambda1 = 0.1,lambda2 = 0.1)
model <- jags.model(model_string, data = data, inits=inits, n.chains=2, quiet=TRUE)
update(model, 10000, progress.bar= "none")
params  <- c("lambda1","lambda2","ratio")

samples <- coda.samples(model,
           variable.names=params,
           n.iter=10000, progress.bar= "none")


plot(samples)
```




```
summary(samples)
```

```
##
## Iterations = 11001:21000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 10000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean       SD  Naive SE Time-series SE
## lambda1 0.2001 0.008462 5.983e-05      8.036e-05
## lambda2 0.4054 0.122344 8.651e-04      1.185e-03
## ratio   2.0290 0.618975 4.377e-03      5.976e-03
##
## 2. Quantiles for each variable:
##
##           2.5%    25%    50%    75%  97.5%
## lambda1 0.1839 0.1944 0.2001 0.2058 0.2171
## lambda2 0.2002 0.3187 0.3930 0.4791 0.6781
## ratio   0.9961 1.5926 1.9655 2.3974 3.4215
```

```
effectiveSize(samples)
```

```
##  lambda1  lambda2    ratio
## 11089.76 10657.90 10731.37
```

```
gelman.diag(samples)
```

```
## Potential scale reduction factors:
##
##         Point est. Upper C.I.
## lambda1          1          1
## lambda2          1          1
## ratio            1          1
##
## Multivariate psrf
##
## 1
```

Convergence: The Gel statistic are all exactly 1 and the effective sample sizes are also large (10K+). The trace plots also look good with a "barcode" shape and don't converge to a single point. All of this points to converging chains.

4.1

```
  # Setting up data for jags
  nplace <- ntreat <- 6
  placebo <- c(2.0, -3.1, -1.0, 0.2, 0.3, 0.4)
  treat    <- c(-3.5, -1.6, -4.6, -0.9, -5.1, 0.1)

  meanplacebo <- mean(placebo)
  meantreat <- mean(treat)

  s2pl   <- mean((placebo - meanplacebo)^2)
  s2trt   <- mean((treat - meantreat)^2)
  sp    <- sqrt((s2pl/2 + s2trt/2))

  postmean <- meantreat - meanplacebo
  postsd <- sp * sqrt( 1/nplace + 1/ntreat)
  credset <- postmean + postsd * qt(c(0.025,0.975), df = nplace + ntreat)

  print(postmean)
```

```
## [1] -2.4
```

```
  print(postsd)
```

```
## [1] 1.012423
```

```
  print(credset)
```

```
## [1] -4.6058799 -0.1941201
```

Difference

```
  data <- list(n = 6, placebo = placebo, treat = treat)

  model_string <- textConnection("model{

   # Likelihood
```

```
  for(i in 1:n){
    placebo[i] ~ dnorm(mu ,tau)
    treat[i] ~ dnorm( mu + delta ,tau)
  }

  # Priors
  mu    ~  dnorm(0, 0.0001)
  delta ~  dnorm(0, 0.0001)
  tau   ~  dgamma(0.1, 0.1)
  sigma <- 1/sqrt(tau)
}")

model <- jags.model(model_string, data = data, n.chains = 2, quiet = TRUE)
update(model, 10000, progress.bar="none")
params  <- c("delta")
samples <- coda.samples(model,
           variable.names=params,
           n.iter=50000, progress.bar="none")
summary(samples)
```

```
##
## Iterations = 10001:60000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 50000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean             SD       Naive SE Time-series SE
##      -2.392705       1.238827       0.003918       0.006828
##
## 2. Quantiles for each variable:
##
##    2.5%     25%     50%     75%   97.5%
## -4.8388 -3.1724 -2.3934 -1.6140  0.0815
```

Delta is probably negative (quantile shows around 95% of the values for delta are negative). Since delta is negative that means the treatment succeeds in decreasing the patients' weight.

Changing the prior results in small changes in the data that would not affect the interpretation. It would simply increase the values' extremity or make it so around 100% of values are negative.

Sensitive priors

```
data <- list(n = 6, placebo = placebo, treat = treat)

model_string <- textConnection("model{

 # Likelihood
 for(i in 1:n){
   placebo[i] ~ dnorm(mu ,tau)
   treat[i] ~ dnorm( mu + delta ,tau)
 }
```

```
  # Priors, can change dnorms mean to 1, increase sd. Also can change dgamma shape and scale
  # dnorm: mean: 1,4 0,1
  # dgamma: 1,1 10,10
  mu    ~  dnorm(0, 0.0001)
  delta ~  dnorm(0, 0.0001)
  tau   ~  dgamma(0.1, 0.1)
  sigma <- 1/sqrt(tau)
}")

model <- jags.model(model_string, data = data, n.chains = 2, quiet = TRUE)
update(model, 10000, progress.bar="none")
params  <- c("delta")
samples <- coda.samples(model,
           variable.names=params,
           n.iter=50000, progress.bar="none")
summary(samples)
```

```
##
## Iterations = 10001:60000
## Thinning interval = 1
## Number of chains = 2
## Sample size per chain = 50000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean             SD       Naive SE Time-series SE
##      -2.400664       1.234124       0.003903       0.006811
##
## 2. Quantiles for each variable:
##
##     2.5%      25%      50%      75%    97.5%
## -4.86567 -3.17450 -2.40054 -1.63282  0.07111
```

4.2

Boston Data

```
library(MASS)
data(Boston)
?Boston
```

```
## starting httpd help server ... done
```

More set up

```
# Data setup
crim <- Boston[,1]
zn <- Boston[,2]
indus <- Boston[,3]
chas <- Boston[,4]
nox <- Boston[,5]
```

```
rm <- Boston[,6]
age <- Boston[,7]
dis <- Boston[,8]
rad <- Boston[,9]
tax <- Boston[,10]
ptratio <- Boston[,11]
black <- Boston[,12]
lstat <- Boston[,13]
medv <- Boston[,14]
```

Scaling and na values

```
X <- cbind(crim, zn, indus,
           chas, nox, rm, age, dis, rad,
           tax, ptratio, black,
           lstat, medv)

X <- as.matrix(scale(X))

# Missing data
miss <- is.na(rowSums(X))
X <- X[!miss,]
medv <- medv[!miss]

# Names
vname <- c("Intercept", "crim", "zn", "indus",
           "chas", "nox", "rm", "age", "dis", "rad",
           "tax", "ptratio", "black",
           "lstat", "medv")

n <- length(medv)
p <- ncol(X)
data2 <- list(Y = medv, X = X, n = n, p = p)
params <- c("alpha", "beta")
```

Jags for 4.2a

```
# Jags
model_string2 <- textConnection("model {

  # Likelihood
  for (i in 1:n) {
    Y[i] ~ dnorm(alpha + inprod(X[i,],beta[]),taue)
  }
  # Priors
    for(j in 1:p){
      beta[j] ~ dnorm(0, 0.001)
    }
    alpha ~ dnorm(0, 0.001)
    taue  ~ dgamma(0.1, 0.1)
}")
```

```
model2 <- jags.model(model_string2, data = data2, n.chains = 2,quiet = TRUE)
update(model2, 10000, progress.bar = "none")
samples2 <- coda.samples(model2, variable.names = params,thin = 10, n.iter = 20000, progress.bar = "none
sum <- summary(samples2)
rownames(sum$statistics) <- vname
rownames(sum$quantiles)  <- vname
effectiveSize(samples2)
```

```
##    alpha  beta[1]  beta[2]  beta[3]  beta[4]  beta[5]  beta[6]  beta[7]
## 4000.000 4000.000 4296.562 3823.296 4144.934 4330.255 4180.978 4033.002
##  beta[8]  beta[9] beta[10] beta[11] beta[12] beta[13] beta[14]
## 4166.462 2401.409 2173.925 3791.355 4282.037 4150.111 4000.000
```

```
gelman.diag(samples2)
```

```
## Potential scale reduction factors:
##
##          Point est. Upper C.I.
## alpha             1       1.00
## beta[1]           1       1.00
## beta[2]           1       1.00
## beta[3]           1       1.00
## beta[4]           1       1.00
## beta[5]           1       1.01
## beta[6]           1       1.00
## beta[7]           1       1.00
## beta[8]           1       1.00
## beta[9]           1       1.00
## beta[10]          1       1.00
## beta[11]          1       1.01
## beta[12]          1       1.01
## beta[13]          1       1.00
## beta[14]          1       1.00
##
## Multivariate psrf
##
## 1
```

```
sum
```

```
##
## Iterations = 10010:30000
## Thinning interval = 10
## Number of chains = 2
## Sample size per chain = 2000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##                 Mean        SD  Naive SE Time-series SE
## Intercept  2.253e+01 0.0008907 1.408e-05      1.408e-05
## crim      -8.029e-06 0.0012015 1.900e-05      1.900e-05
## zn        -1.312e-05 0.0013748 2.174e-05      2.100e-05
## indus     -6.999e-06 0.0018082 2.859e-05      2.929e-05
```

```
## chas      -1.096e-05 0.0009521 1.505e-05      1.480e-05
## nox       -1.156e-05 0.0019293 3.051e-05      2.931e-05
## rm         6.931e-06 0.0013283 2.100e-05      2.057e-05
## age       -5.573e-06 0.0015688 2.480e-05      2.469e-05
## dis       -1.299e-05 0.0018641 2.947e-05      2.887e-05
## rad        3.088e-05 0.0024995 3.952e-05      5.100e-05
## tax        4.627e-06 0.0027348 4.324e-05      5.871e-05
## ptratio   -6.366e-05 0.0012760 2.018e-05      2.075e-05
## black      3.476e-05 0.0010430 1.649e-05      1.598e-05
## lstat     -2.464e-05 0.0017064 2.698e-05      2.650e-05
## medv       9.197e+00 0.0017345 2.742e-05      2.743e-05
##
## 2. Quantiles for each variable:
##
##                2.5%        25%        50%       75%     97.5%
## Intercept 22.531068 22.5322054  2.253e+01 2.253e+01 22.534593
## crim      -0.002379 -0.0008272 -4.391e-06 8.023e-04  0.002340
## zn        -0.002684 -0.0009367 -3.705e-05 9.099e-04  0.002769
## indus     -0.003476 -0.0012170 -2.414e-05 1.234e-03  0.003563
## chas      -0.001915 -0.0006561 -3.243e-06 6.298e-04  0.001849
## nox       -0.003825 -0.0013071  8.845e-06 1.272e-03  0.003803
## rm        -0.002674 -0.0008895  1.128e-05 9.069e-04  0.002590
## age       -0.002992 -0.0010643 -6.199e-05 1.008e-03  0.003138
## dis       -0.003663 -0.0013060  2.867e-05 1.290e-03  0.003594
## rad       -0.004743 -0.0016556  3.485e-06 1.680e-03  0.005061
## tax       -0.005518 -0.0018595  5.565e-06 1.887e-03  0.005271
## ptratio   -0.002610 -0.0009104 -4.605e-05 7.737e-04  0.002413
## black     -0.001989 -0.0006654  5.803e-05 7.233e-04  0.002048
## lstat     -0.003257 -0.0011759 -2.808e-05 1.138e-03  0.003283
## medv       9.193683  9.1959083  9.197e+00 9.198e+00  9.200510
```

Looks like the dist converges as Gel stat is exactly 1 for most and the effect sample size is 1k+, cant graph as the margins are too large. Could run longer. The summaries are in the quantiles and means.

LM for 4.2b

```
modellm <- lm( medv ~
            scale(crim) + scale(zn) + scale(indus) +
           scale(chas) + scale(nox) + scale(rm) + scale(age) + scale(dis) + scale(rad) +
           scale(tax) + scale(ptratio) + scale(black) +
           scale(lstat), data = data2)
summary(modellm)
```

```
##
## Call:
## lm(formula = medv ~ scale(crim) + scale(zn) + scale(indus) +
##     scale(chas) + scale(nox) + scale(rm) + scale(age) + scale(dis) +
##     scale(rad) + scale(tax) + scale(ptratio) + scale(black) +
##     scale(lstat), data = data2)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -15.595  -2.730  -0.518   1.777  26.199
##
```

```
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)    22.53281    0.21095 106.814  < 2e-16 ***
## scale(crim)    -0.92906    0.28269  -3.287 0.001087 **
## scale(zn)       1.08264    0.32016   3.382 0.000778 ***
## scale(indus)    0.14104    0.42188   0.334 0.738288
## scale(chas)     0.68241    0.21884   3.118 0.001925 **
## scale(nox)     -2.05875    0.44262  -4.651 4.25e-06 ***
## scale(rm)       2.67688    0.29364   9.116  < 2e-16 ***
## scale(age)      0.01949    0.37184   0.052 0.958229
## scale(dis)     -3.10712    0.41999  -7.398 6.01e-13 ***
## scale(rad)      2.66485    0.57770   4.613 5.07e-06 ***
## scale(tax)     -2.07884    0.63379  -3.280 0.001112 **
## scale(ptratio) -2.06265    0.28323  -7.283 1.31e-12 ***
## scale(black)    0.85011    0.24521   3.467 0.000573 ***
## scale(lstat)   -3.74733    0.36216 -10.347  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.745 on 492 degrees of freedom
## Multiple R-squared:  0.7406, Adjusted R-squared:  0.7338
## F-statistic: 108.1 on 13 and 492 DF,  p-value: < 2.2e-16
```

Results are different . The Bayesian model attempts to find the best interval for each parameter and estimate a posterior distribution with it. While the lm function tries to minimize squares to fit a curve. Should lead to the same results.

Double exp priors for 4.2c

```
model_stringbl <- textConnection("model{
  # Likelihood
   for(i in 1:n){
     Y[i] ~ dnorm(alpha+inprod(X[i,],beta[]),taue)
   }
  # Priors
   for(j in 1:p){
     beta[j] ~ ddexp(0,taue*taub)
   }
   alpha ~ dnorm(0,0.001)
   taue  ~ dgamma(0.1, 0.1)
   taub  ~ dgamma(0.1, 0.1)
}")

model3 <- jags.model(model_stringbl, data = data2, n.chains = 2,quiet = TRUE)
update(model3, 10000, progress.bar = "none")
samples3 <- coda.samples(model3, variable.names = params, thin = 10, n.iter = 20000, progress.bar = "no
```

Summary of 4.2c

```
sumbl <- summary(samples3)
rownames(sumbl$statistics) <- vname
rownames(sumbl$quantiles)  <- vname
```

```
sumbl
```

```
##
## Iterations = 11010:31000
## Thinning interval = 10
## Number of chains = 2
## Sample size per chain = 2000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##                 Mean        SD  Naive SE Time-series SE
## Intercept  2.253e+01 0.0008964 1.417e-05      1.417e-05
## crim       1.478e-05 0.0012175 1.925e-05      1.962e-05
## zn        -5.169e-07 0.0013826 2.186e-05      2.263e-05
## indus     -2.005e-05 0.0017658 2.792e-05      2.957e-05
## chas      -1.594e-05 0.0009405 1.487e-05      1.487e-05
## nox        2.119e-05 0.0019041 3.011e-05      3.351e-05
## rm        -6.913e-06 0.0013443 2.126e-05      2.179e-05
## age       -2.129e-05 0.0015651 2.475e-05      2.547e-05
## dis        4.072e-06 0.0018947 2.996e-05      3.426e-05
## rad       -2.965e-05 0.0025322 4.004e-05      6.066e-05
## tax        3.247e-05 0.0027060 4.279e-05      6.608e-05
## ptratio   -6.908e-06 0.0012850 2.032e-05      2.032e-05
## black      3.219e-06 0.0010555 1.669e-05      1.662e-05
## lstat      1.074e-05 0.0016879 2.669e-05      2.853e-05
## medv       9.197e+00 0.0017925 2.834e-05      3.420e-05
##
## 2. Quantiles for each variable:
##
##                2.5%        25%        50%       75%     97.5%
## Intercept 22.531058 22.5321974  2.253e+01 2.253e+01 22.534534
## crim      -0.002360 -0.0008311  2.223e-05 8.267e-04  0.002360
## zn        -0.002799 -0.0008992  2.871e-05 9.245e-04  0.002690
## indus     -0.003533 -0.0011861 -1.708e-05 1.148e-03  0.003463
## chas      -0.001827 -0.0006519 -1.489e-05 5.988e-04  0.001864
## nox       -0.003743 -0.0012813  2.353e-05 1.292e-03  0.003744
## rm        -0.002656 -0.0009161 -8.599e-06 8.991e-04  0.002642
## age       -0.003097 -0.0010839 -2.441e-05 1.039e-03  0.003043
## dis       -0.003643 -0.0012687  3.195e-05 1.362e-03  0.003657
## rad       -0.005046 -0.0017297 -1.069e-06 1.655e-03  0.004900
## tax       -0.005248 -0.0018341  1.662e-05 1.824e-03  0.005417
## ptratio   -0.002513 -0.0008599 -1.192e-05 8.687e-04  0.002498
## black     -0.002077 -0.0007111  3.065e-05 7.009e-04  0.002067
## lstat     -0.003266 -0.0010891 -2.436e-05 1.128e-03  0.003396
## medv       9.193742  9.1958763  9.197e+00 9.198e+00  9.200693
```

The results are smaller, but as the uniformative is close to zero the results aren't too different. This is different because the BLASSO, BL, model allows for the coef to be set to basically zero, while the gaussian are usually larger.

Fitting 4.2a again for 4.2d

```
Yo    <- medv[1:500]    # Observed data
Xo    <- X[1:500,]

Yp    <- medv[501:506]     # Counties set aside for prediction
Xp    <- X[501:506,]
no    <- length(Yo)
np    <- length(Yp)
data4 <- list(Yo=Yo,no=no,np=np,p=p,Xo=Xo,Xp=Xp)

params = c("alpha", "beta", "Yp")
model_string4 <- textConnection("model {

  # Likelihood
  for (i in 1:no) {
    Yo[i] ~ dnorm(alpha + inprod(Xo[i,],beta[]),taue)
  }

    # Prediction
  for(i in 1:np){
    Yp[i]  ~ dnorm(mup[i],taue)
    mup[i] <- alpha + inprod(Xp[i,],beta[])
  }
  # Priors
    for(j in 1:p){
      beta[j] ~ dnorm(0, 0.001)
    }
    alpha ~ dnorm(0, 0.001)
    taue  ~ dgamma(0.1, 0.1)
}")
model4 <- jags.model(model_string4, data = data4, n.chains = 2,quiet = TRUE)
update(model4, 10000, progress.bar = "none")
samples4 <- coda.samples(model4, variable.names = params,thin = 10, n.iter = 20000, progress.bar = "none
```

Checking data for s4

```
summary(samples4[,-c(1:np)])
```

```
##
## Iterations = 10010:30000
## Thinning interval = 10
## Number of chains = 2
## Sample size per chain = 2000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##                Mean        SD  Naive SE Time-series SE
## alpha     2.253e+01 0.0009418 1.489e-05      1.489e-05
## beta[1]   1.088e-05 0.0012313 1.947e-05      1.947e-05
## beta[2]  -7.308e-06 0.0014198 2.245e-05      2.188e-05
## beta[3]  -3.445e-05 0.0018243 2.884e-05      2.971e-05
```

```
## beta[4]   1.985e-05 0.0009531 1.507e-05      1.650e-05
## beta[5]  -1.502e-05 0.0019632 3.104e-05      3.105e-05
## beta[6]  -1.652e-05 0.0013725 2.170e-05      2.063e-05
## beta[7]  -1.129e-05 0.0015957 2.523e-05      2.457e-05
## beta[8]  -1.688e-05 0.0019332 3.057e-05      3.123e-05
## beta[9]  -4.647e-05 0.0025439 4.022e-05      4.793e-05
## beta[10]  6.335e-05 0.0027730 4.384e-05      5.236e-05
## beta[11]  1.500e-05 0.0012835 2.029e-05      2.022e-05
## beta[12] -2.171e-05 0.0010675 1.688e-05      1.688e-05
## beta[13] -2.867e-05 0.0017376 2.747e-05      2.712e-05
## beta[14]  9.197e+00 0.0017893 2.829e-05      2.971e-05
##
## 2. Quantiles for each variable:
##
##               2.5%        25%        50%       75%     97.5%
## alpha    22.530919 22.5321858  2.253e+01 2.253e+01 22.534674
## beta[1]  -0.002462 -0.0007985  6.279e-06 8.193e-04  0.002428
## beta[2]  -0.002789 -0.0009662  1.497e-05 9.475e-04  0.002738
## beta[3]  -0.003660 -0.0012488 -4.845e-05 1.221e-03  0.003513
## beta[4]  -0.001808 -0.0006210  1.167e-05 6.557e-04  0.001895
## beta[5]  -0.003879 -0.0013328 -4.785e-05 1.314e-03  0.003862
## beta[6]  -0.002629 -0.0009364 -1.268e-05 8.795e-04  0.002704
## beta[7]  -0.003148 -0.0010839 -1.632e-05 1.055e-03  0.003100
## beta[8]  -0.003908 -0.0012759 -1.762e-05 1.265e-03  0.003769
## beta[9]  -0.004970 -0.0017516 -2.649e-05 1.696e-03  0.004953
## beta[10] -0.005430 -0.0017435  8.836e-05 1.928e-03  0.005505
## beta[11] -0.002537 -0.0008507  3.575e-05 8.687e-04  0.002519
## beta[12] -0.002167 -0.0007249 -1.947e-05 7.095e-04  0.002037
## beta[13] -0.003433 -0.0011862 -1.782e-05 1.121e-03  0.003391
## beta[14]  9.193576  9.1958922  9.197e+00 9.198e+00  9.200590
```

Plotting ppd

```
#Extract the samples for each parameter

 samples4s <- samples4[[1]]
 Yp.samples4s <- samples4s[,1:np]
 alpha.samples4s <- samples4s[,np+1]
 beta.samples4s <- samples4s[,np+1+1:p]

# Compute the posterior mean for the plug-in predictions

 beta.mn <- colMeans(beta.samples4s)
 alpha.mn <- mean(alpha.samples4s)

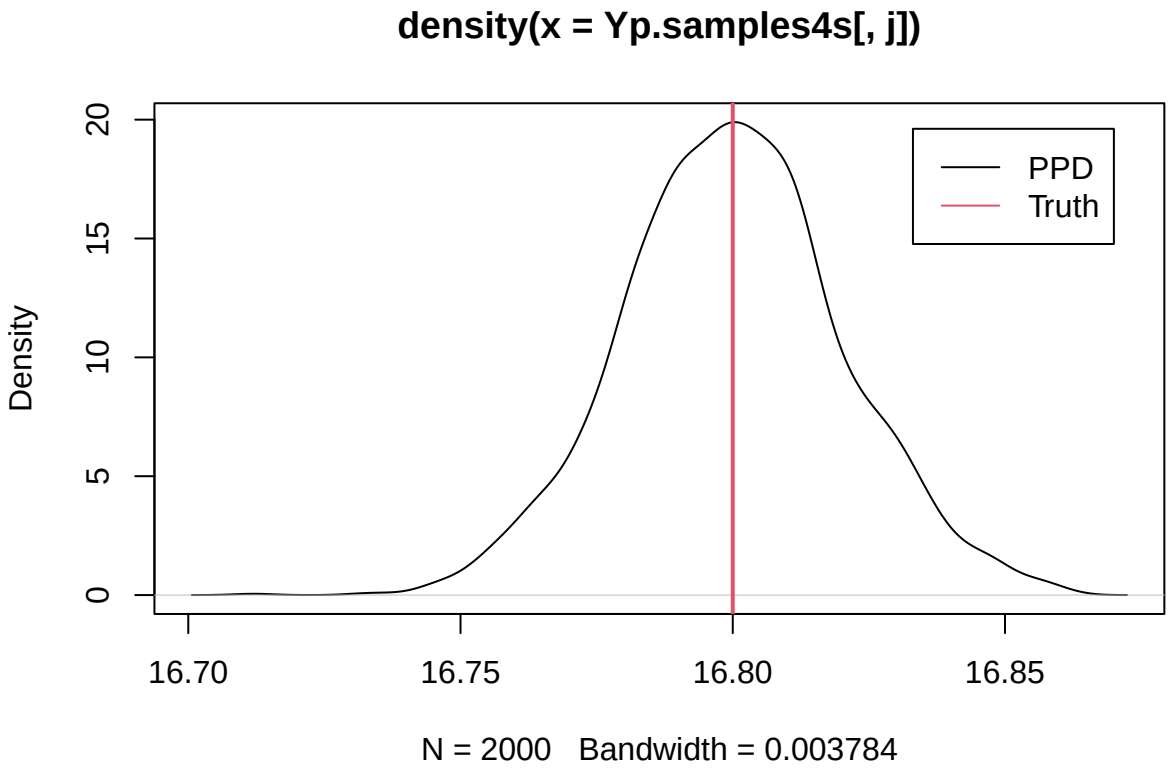

# Plot the PPD and plug-in

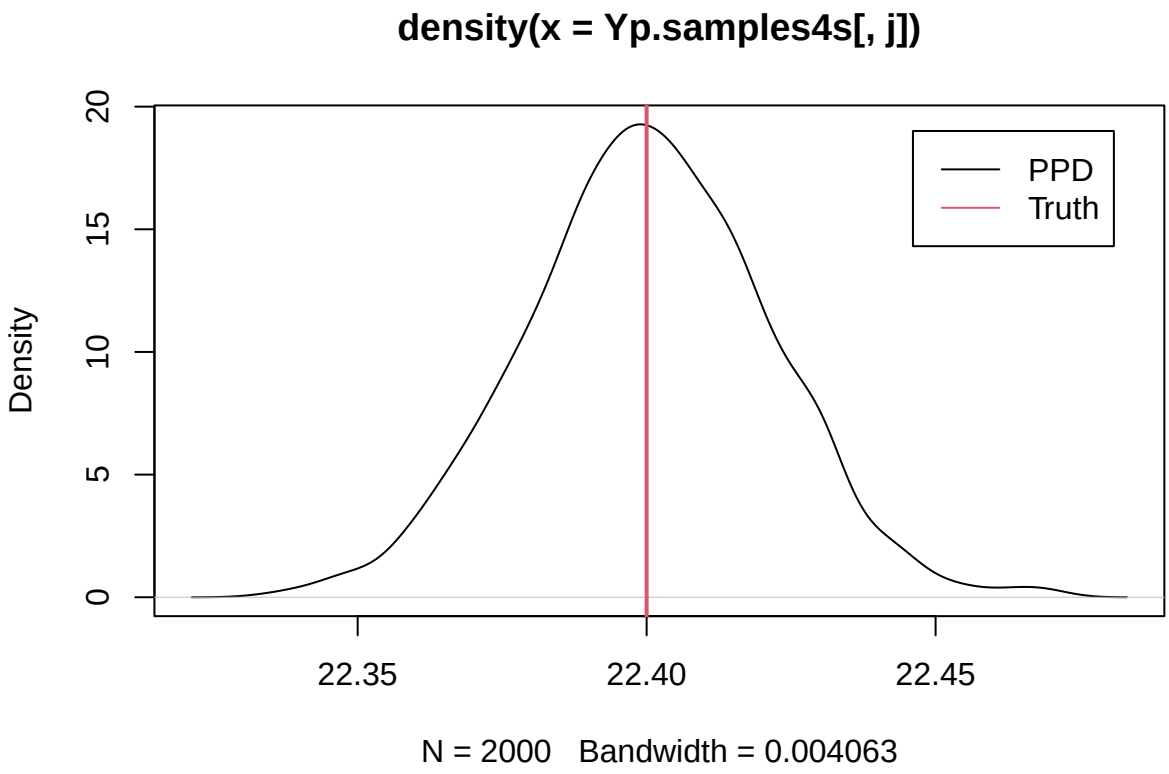
 for(j in 1:6){

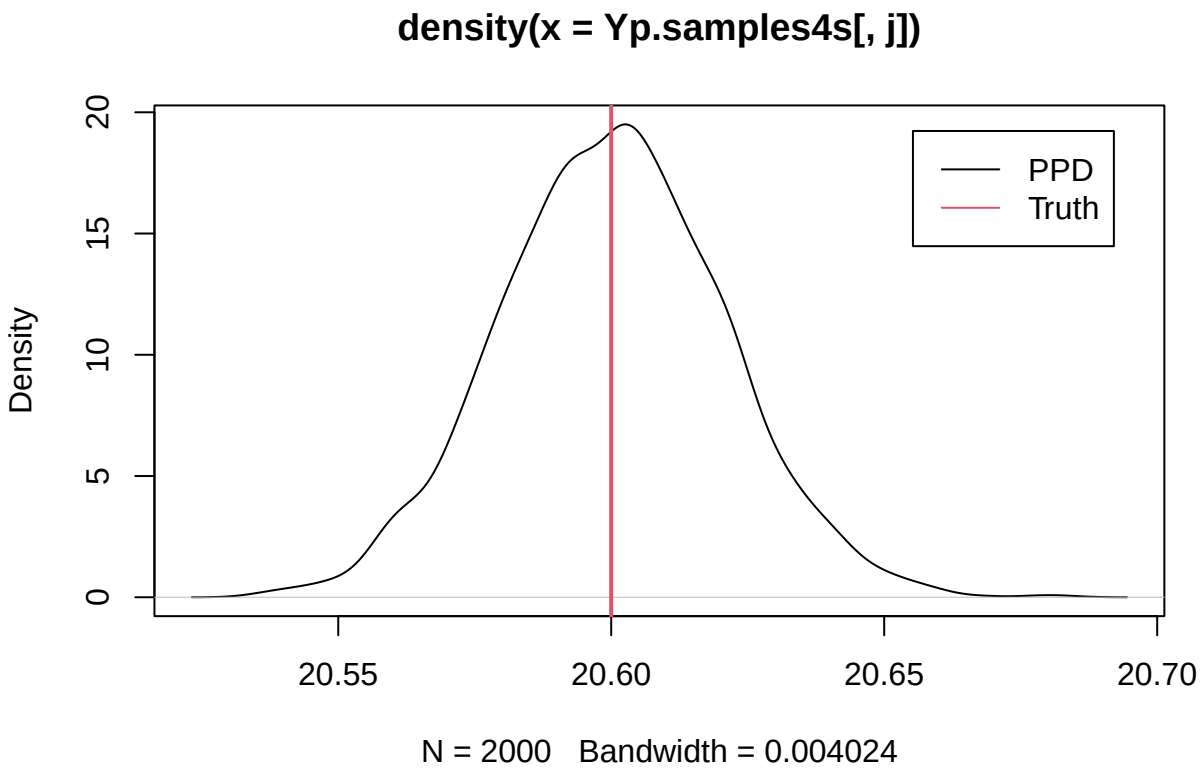
    # PPD
    plot(density(Yp.samples4s[,j]))
```

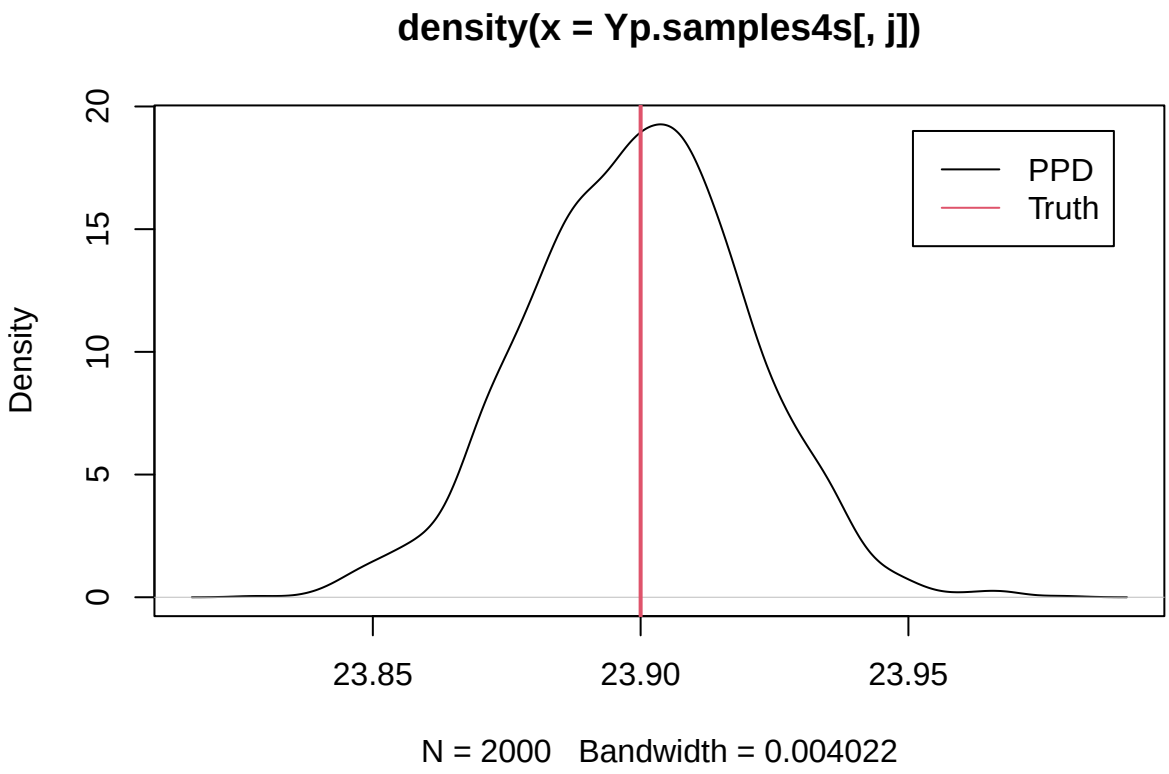
```
    # Truth
    abline(v=Yp[j],col=2,lwd=2)

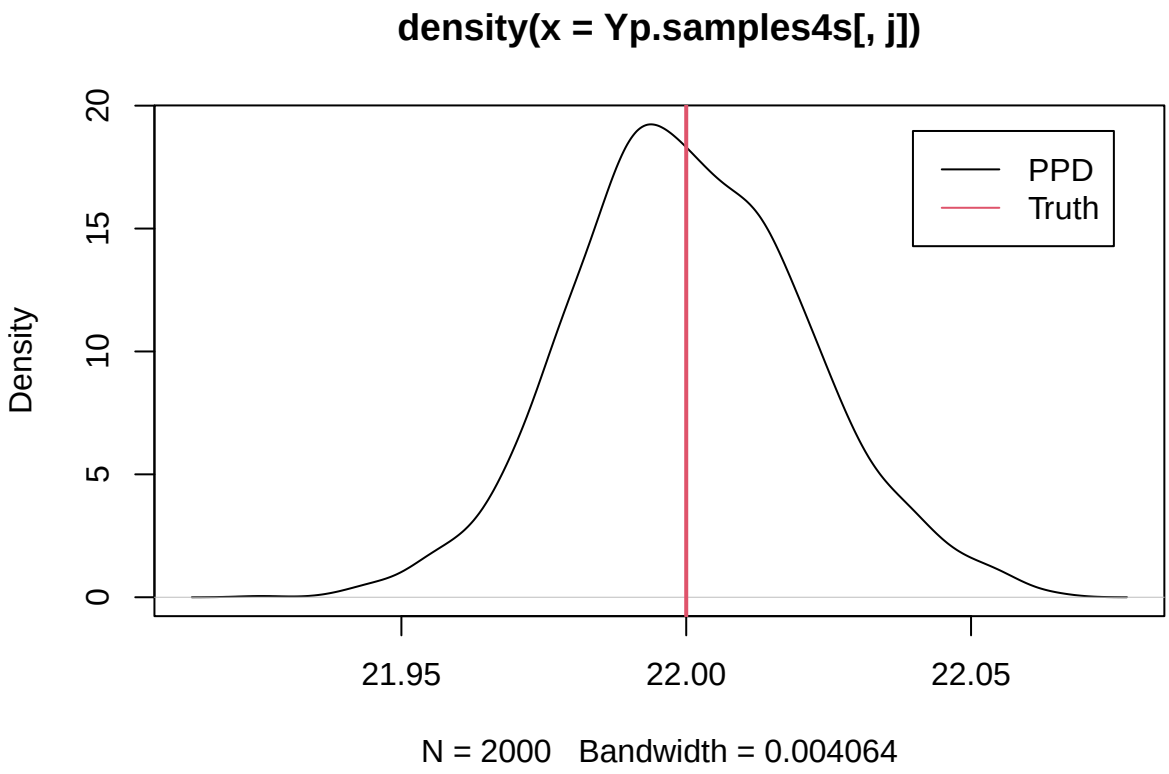
    legend("topright",c("PPD","Truth"),col=1:2,lty=1,inset=0.05)
}
```

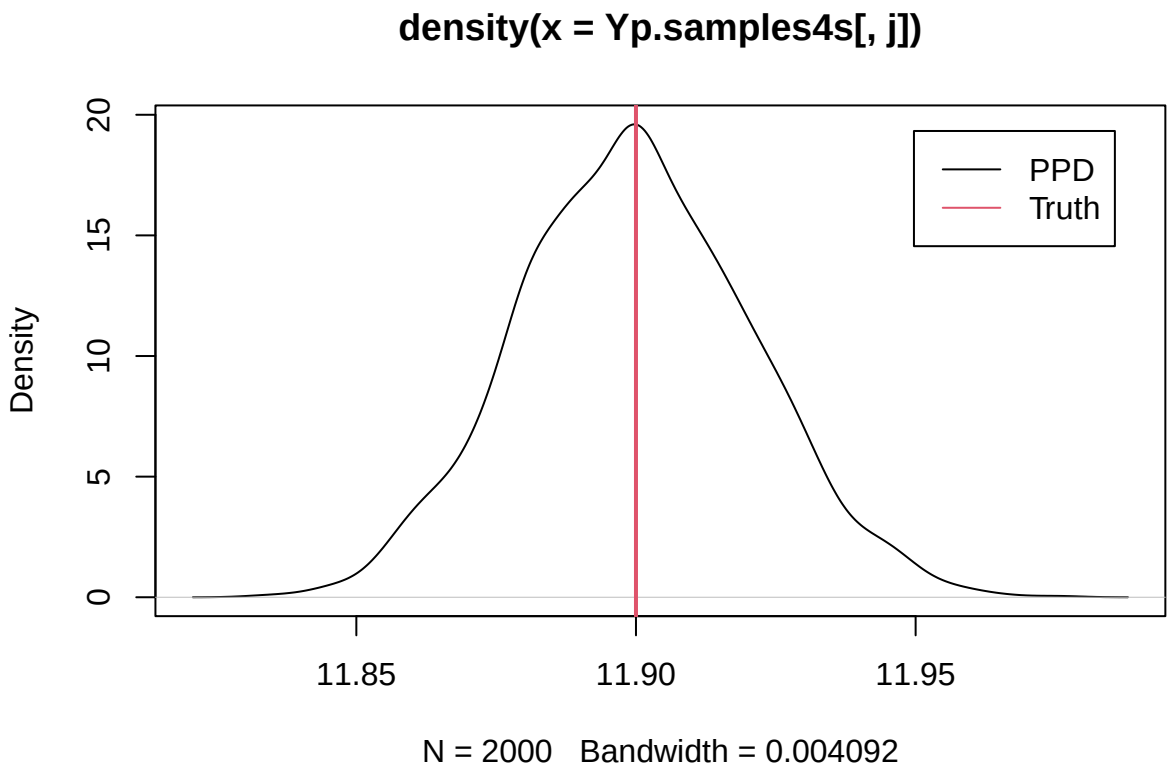



density(x = Yp.samples4s[, j])
Density
0
5
10
15
20
22.35
22.40
22.45
PPD
Truth
N = 2000 Bandwidth = 0.004063


density(x = Yp.samples4s[, j])
Density
0
5
10
15
20
20.55
20.60
20.65
20.70
PPD
Truth
N = 2000 Bandwidth = 0.004024


density(x = Yp.samples4s[, j])
Density
0
5
10
15
20
23.85
23.90
23.95
PPD
Truth
N = 2000 Bandwidth = 0.004022


density(x = Yp.samples4s[, j])
Density
0
5
10
15
20
21.95
22.00
22.05
PPD
Truth
N = 2000 Bandwidth = 0.004064



Looks reasonable as the peak of the densities are around of the abline of the true values.